

codex-windows / 019f2179-d9ee-7da3-9421-1f409958cad9

Metadata

```
- Source: `codex-windows`  
- Kind: `codex`  
- Source file: `C:\Users\avidu\.codex\sessions\2026\07\02\rollout-2026-07-02T11-47-41-019f2179-d9ee-7da3-9421-1f409958cad9.jsonl`  
- SHA-256: `1ebe66d317ccd2cbe9be5c1267617df88873de6a3646f80614fcf51ecc5c1a96`  
- Source modified: `2026-07-02T06:40:43+00:00`  
- Imported at: `2026-07-05T16:48:16+00:00`  
- cli_version: `0.142.5`  
- cwd: `C:\Users\avidu\Projects\khelsutra-guru`  
- model_provider: `openai`  
- session_id: `019f2179-d9ee-7da3-9421-1f409958cad9`  
- source: `vscode`
```

Transcript

1. developer (2026-07-02T06:19:25.633Z)

```
<permissions instructions>  
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled. Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason, commands will be rejected.  
</permissions instructions>  
<app-context>
```

Codex desktop context

- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files

- In the app, the model can display images and videos using standard Markdown image syntax: `![alt](url)`
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., `![alt](/absolute/path.png)`); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., `[label](https://example.com)`).

Workspace Dependencies

- For sheets, slides, and documents, call ``load_workspace_dependencies`` to find the bundled runtime and libraries.

Automations

- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the ``automation_update`` tool first, then follow its schema instead of writing raw automation directives by hand.

- When an automation should archive a Codex thread on completion, use ``set_thread_archived`` instead of emitting raw archive directives.

Thread Coordination

- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: ``create_thread``, ``fork_thread``, ``list_threads``, ``read_thread``, ``send_message_to_thread``, ``handoff_thread``,

`set\_thread\_pinned`, `set\_thread\_archived`, or `set\_thread\_title`.

- Only use `create\_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create\_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments

- Use the `::code-comment{...}` directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: `::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}`

```
</app-context>
<collaboration_mode># Collaboration Mode: Default
```

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different ``<collaboration_mode>...</collaboration_mode>`` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request\_user\_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

```
</collaboration_mode>
<apps_instructions>
```

Apps (Connectors)

Apps (Connectors) can be explicitly triggered in user messages in the format `[App-name](app://{connector\_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.

An app is equivalent to a set of MCP tools within the `codex\_apps` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool\_search` tool. If `tool\_search` is available, the apps that are searchable by `tools\_search` will be listed by it.

Do not additionally call `list\_mcp\_resources` or `list\_mcp\_resource\_templates` for apps.

```
</apps_instructions>
<skills_instructions>
```

Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

Skill roots

- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`

- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.5/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.3/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.3/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.7/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the output should be a bitmap asset rather than repo-native code or vector. Do not use when the task is better handled by editing e (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex docs questions, use the Codex manual helper first for broad Codex self-knowledge, and restrict fallback browsing to official (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace entries for plugin ordering and availability metadata, or update an existing local plugin during development with the C (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into \$CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for readers. Trigger when user mentions writing docs, creating proposals, drafting specs, or similar documentation tasks. (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also use when extracting or reorganizing content from .docx files, inserting or replacing images in documents, performing find-an (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting images, and OCR on scanned PDFs to make them searchable. If the user mentions a .pdf file or asks to produce one, use this (file: r2/pdf/SKILL.md)
- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, modifying, or updating existing presentations; combining or splitting slide files; working with templates, layouts, speaker (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)

- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggering accuracy. (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from scratch or from other data sources; or convert between tabular file formats. Trigger especially when the user references a spr (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.81905/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.81905/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.81905/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for delivering the final .plugin file. (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on .docx, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render\_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final document. (file: r12/documents/26.630.12135/skills/documents/SKILL.md)

- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whenever thread-level state, resolution status, or inline review context matters. (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies or forwarded messages, or organize messages with explicit confirmation before send, archive, delete, or label actions. (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)

- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or cancel changes with timezone-aware details. (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and reference routing for formatting, citations, tables, and write-safety. (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r12/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r12/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)
- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-

creator/SKILL.md)

- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r12/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes \$template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed artifact-template skill. Do not use for one-off artifact creation from an existing template. (file: r12/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `\$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
 - 1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
 - 2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
 - 3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
 - 4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
 - 5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
 - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
 - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
 - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
 - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
 - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.

</skills_instructions>

<plugins_instructions>

Plugins

A plugin is a local bundle of skills, MCP servers, and apps.

How to use plugins

- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin\_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp\_server\_tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
- Missing/blocked: If the user requests a plugin that does not have relevant callable

capabilities for the task, say so briefly and continue with the best fallback.
</plugins_instructions>

2. user (2026-07-02T06:19:25.634Z)

AGENTS.md instructions

<INSTRUCTIONS>

Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading

****Always `git pull` a repo before starting to read it**** ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- ****Why:**** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- ****How (safe form):**** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- ****If it can't fast-forward**** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts

Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit ****concurrently****, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a ****squash-merge** silently absorbed the sibling's work**. Protocol ? do this EVERY time, not only when you suspect a collision:

- ****Branch from the REMOTE, explicitly:**** `git fetch origin` then `git checkout -b <name> origin/<base>` (e.g. `origin/master`). Never assume the current branch is the intended base.
- ****Re-verify right before `git add`/commit:**** `git branch --show-current` + `git log --oneline -1` (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- ****Stage explicit paths**** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked files can't ride into your commit.
- ****Serialize repo writes:**** don't start a repo-writing spawned/background task while you hold uncommitted work ? commit or push first. If one may be running, treat the tree + current branch as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living ****session handoff**** for each project and use it to resume context quickly across windows.

- ****Where it lives:****
 - If the project has an auto-memory dir (`~/Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a ****"? Start here**** entry at the top of that project's `MEMORY.md`.
 - Otherwise, keep a `SESSION\_HANDOFF.md` at the repo root.
- ****What it contains**** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
 1. ****You are here**** ? current state.
 2. ****Next steps / open threads.****
 3. ****Ramp-up kit**** ? the exact files/docs to read to get current.

4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a restart can ramp up without losing context. Do **not** rewrite it every turn ? only when something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally **verified** the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run_all_tests.py` / `pytest``), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .`` and `mypy ...``, matching the repo's CI. Install `ruff`/`mypy`` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under`` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a **verified** stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.

</INSTRUCTIONS>

<environment_context>

```
<cwd>C:\Users\avidu\Projects\khelsutra-guru</cwd>
<shell>powershell</shell>
<current_date>2026-07-02</current_date>
<timezone>Asia/Calcutta</timezone>
<filesystem><workspace_roots><root>C:\Users\avidu\Projects\khelsutra-
guru</root></workspace_roots><permission_profile type="disabled"><file_system
type="unrestricted" /></permission_profile></filesystem>
</environment_context>
```

3. user (2026-07-02T06:19:25.668Z)

Hello. Can you build up a deep understanding of the project khelsutra-guru which is located at Users/avidu/Projects/khelsutra-guru and cross check the audit report docs/AUDIT_REPORT_khelsutra-guru_2026-07-02.md to identify the completed items, ensure its is tracked as a project per the repo conventions and then give me a plan of your execution to complete the work and bring this repo in a healthy state. Feel free to push back on some audit findings or add more as you identify while reading the code.

4. assistant (2026-07-02T06:19:34.978Z)

I'll start by syncing the checkout in the safe repo convention way, then I'll read the handoff/audit/docs/code paths needed to separate already-completed work from true remaining risks. After that I'll give you a concrete execution plan, including any audit items I'd challenge or add.

5. assistant (2026-07-02T06:19:49.129Z)

The first convention check found that `C:\Users\avidu\Projects\khelsutra-guru`` itself is not currently a Git working tree. I'm going to locate the actual checkout inside/near it before reading project files, so I can still honor the pull-first rule where it applies.

6. assistant (2026-07-02T06:20:02.559Z)

This workspace is a multi-checkout project folder rather than a single repo. I found five Git repos under it, so I'm going to fast-forward-pull each one before reading their contents.

7. user (2026-07-02T06:20:05.642Z)

btw make sure to sync to repos to head

8. assistant (2026-07-02T06:20:17.846Z)

Yep, that's exactly what I'm doing: every Git checkout under the project folder has been pulled with `--ff-only`. I'm checking the resulting branch/status now so I can catch any checkout that is still stale, dirty, or on an unexpected branch before I rely on it.

9. assistant (2026-07-02T06:20:29.459Z)

All five repos are now at their remote tracking branches; `sports-data-collector` has a pre-existing modified `.gitignore`, so I'll treat that as someone else's WIP and avoid touching it. Next I'm locating the project handoff/memory files and the canonical audit report copy.

10. assistant (2026-07-02T06:20:46.155Z)

I found the audit report in two places: a root-level copy and `badminton-highlight-indexer/docs/`. The pulled repo added the `docs/` copy, so I'm going to treat that as the canonical tracked audit unless the file hashes disagree. I also didn't find an existing project memory/handoff yet, which is already one repo-convention gap to close.

11. assistant (2026-07-02T06:21:03.690Z)

The project's local memory is in `badminton-highlight-indexer/CLAUDE.md`, and it says substantial project work should have a tracked doc using the project-doc template. I'm reading the audit in structured chunks now, then I'll verify findings against the current post-pull code rather than trusting stale lines.

12. assistant (2026-07-02T06:21:16.357Z)

The audit is dense but useful: it has both confirmed findings and verifier pushback already embedded. I've got the high-level map; now I'm filling in the middle sections that were too large for one terminal readout and checking what the recent fast-forwards already changed.

13. user (2026-07-02T06:21:17.366Z)

also feel free to read Onedrive/Documents/claude-sync to understand the coding conventions and very tight rules for operating in the repo

14. assistant (2026-07-02T06:21:28.638Z)

Good call. I'll pull in the `claude-sync` operating rules before I finalize the execution plan; if that folder is itself a Git checkout I'll fast-forward it first, same convention.

15. assistant (2026-07-02T06:21:34.822Z)

`claude-sync` exists and is not a Git repo, so there's no pull step to perform. I'm going to read its index/rule files selectively and fold those constraints into the health plan.

16. assistant (2026-07-02T06:21:48.650Z)

There is an active `memory/khelsutra-guru/` tree in `claude-sync`, including the exact handoff and working-agreement notes that the repo convention was pointing toward. I'm reading the ?start here? memory and the operating agreements now; this is the missing tracking layer for the project.

17. assistant (2026-07-02T06:22:04.123Z)

One important thing already emerged: the audit report was verified against pre-pull SHAs, but the pulls brought in fresh fixes in four repos. So the next step is a delta audit: compare each repo's new commits against the audit findings and mark what is now complete, still open, or newly suspect.

18. assistant (2026-07-02T06:22:37.160Z)

The memory layer says this cleanup was already recognized as a multi-repo project on July 2, but the durable repo-side tracking still needs to be verified against the code/docs after the latest pulls. I'm checking the fresh commits now so I can distinguish 'already fixed upstream' from 'still needs a PR'.

19. assistant (2026-07-02T06:22:49.860Z)

The delta is nice and concrete: new commits explicitly target several audit IDs: engine `R1-19/R1-20/R1-27`, engine docs truth-sync, khelsutra retired-droplet docs, rally-annotator `R2-1/R2-2`, and sdc `R2-3`. I'm verifying those by reading the actual diffs and current code, because commit messages are friendly but not proof.

20. assistant (2026-07-02T06:23:05.055Z)

One commit is worth a closer look: the engine `gcs.py` fix claims it collapsed the import, but the diff still leaves the ignore inside a parenthesized import. Since the original bug was line-position-sensitive mypy behavior, I'm going to verify that one instead of marking it complete from the message alone.

21. assistant (2026-07-02T06:23:09.421Z)

Both `mypy` and `google-cloud-storage` are installed locally, so this is verifiable on this box. Running the repo's broad mypy gate now to see whether `R1-27` is actually closed or only half-fixed.

22. assistant (2026-07-02T06:24:37.186Z)

The `R1-27` fix is not actually complete: the same full mypy gate still fails at `backend/storage/gcs.py:31`. I'll treat that as a still-open audit item with a failed attempted fix, not as completed.

23. assistant (2026-07-02T06:24:42.922Z)

A few local hygiene items from the audit are already resolved in this checkout: the nested `sports-data-collector/sports-data-collector` clone is gone, the engine `.git` bloat has been pruned down to about 11 MB packed, and the code repos are otherwise clean except the pre-existing `sports-data-collector/.gitignore` WIP.

24. assistant (2026-07-02T06:24:57.601Z)

The `sports-data-collector/.gitignore` hunk is exactly the stale artifact-policy WIP from the audit, still uncommitted and still contradictory. I won't touch it in this planning pass, but it stays on the execution list with the 'stage explicit paths only' warning in bold ink.

25. assistant (2026-07-02T06:25:25.040Z)

The repo-side tracking is only partial: the audit report is committed under engine `docs/`, and memory `current-state` has a cleanup project entry, but the audit is not indexed in `docs/README.md` or `DOC\_STATUS.md`, and it is not shaped as a tracked project doc with a \$7 progress tracker. That becomes step zero of the execution plan.

26. assistant (2026-07-02T06:25:52.586Z)

I'm separating 'complete' into two kinds: shipped commits on origin, and local-only hygiene

cleanups. That matters because local deletion of untracked debris helps this machine, but it is not the same as a durable repo fix that survives a fresh checkout.

27. assistant (2026-07-02T06:26:14.858Z)

A couple of audit severities need re-grounding now: upload-retention scheduling is still a real gap, but the ?live droplet accepting uploads? premise is no longer true after the droplet retirement docs fix. Conversely, the mypy type-ignore item is worse than it looks because it was marked fixed but still fails locally.

28. assistant (2026-07-02T06:26:28.544Z)

The engine-served SPA bundle is still stale: `backend/ui/ui\_version.json` still pins `ed33a515` from June 22 while `khelsutra/web` has the July 1 feature wave on top. That remains a high-value cross-repo item.

29. assistant (2026-07-02T06:26:43.119Z)

I?m pulling the exact top P1 finding titles that were squeezed out of the earlier long audit read, so the final plan doesn?t hand-wave the highest-risk engine work.

30. assistant (2026-07-02T06:27:12.165Z)

Synced all five repos to remote head with `git pull --ff-only`. The workspace root itself is not a git repo; the canonical tracked audit copy is in [docs/AUDIT_REPORT_khelsutra-guru_2026-07-02.md](<C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/AUDIT_REPORT_khelsutra-guru_2026-07-02.md>). The root-level audit file is a duplicate, outside git.

What Is Already Done

- Engine docs truth-sync landed in `#421`: stale `CLAUDE.md`, `BACKLOG.md`, `DOC\_STATUS.md`, `README.md`, `NEXT\_STEPS.md` items from `R2-5`, `R1-13`, `R1-14`.
- Engine config fixes landed in `#422`: `R1-19` output dir clobber and `R1-20` upload dir app-home resolution.
- Khelsutra retired-droplet doc rot landed in `#142`: `R1-06`, `R1-09`, `R1-36`.
- Rally annotator fixes landed in `#27`: `R2-1` hidden-panel DOM scan and `R2-2` locale persistence.
- Sports data collector fixed `R2-3`: `obsreport` is now direct `git+ssh` from `Khelsutra/sports-obsreport`.
- Local hygiene already done: stale nested `sports-data-collector/sports-data-collector/` is gone; yt-dlp debris is gone; engine `.git` bloat is pruned.

Pushback / Corrections

- `R1-27` is not fixed despite `#422`. I ran `python -m mypy backend/training --no-incremental`; it still fails at `backend/storage/gcs.py:31`.
- The cleanup is only partially tracked per repo convention. Memory `current-state` tracks it, and the audit report is committed, but there is no proper tracked project doc with \$7 progress tracker, `docs/README.md` index entry, and `DOC\_STATUS.md` row.
- `R2-8` upload-retention scheduling is still real, but its ?live droplet accepting uploads? premise is now stale because the droplet is retired. Reclassify around future hosted deployments.
- Add finding: engine `CODE\_MAP.md` is now stale after `#422` around output-dir behavior.

Execution Plan

1. Create a tracked health-remediation project doc from `docs/PROJECT\_DOC\_TEMPLATE.md`, index it in `README`/`DOC\_STATUS`, and use its \$7 as the source of truth.
2. Wave 1: finish failed/cheap correctness fixes: `R1-27`, `R1-02` Gemini re-bill idempotency, `R1-03/R1-17` remote-source scratch/cache leak, ffmpeg subprocess timeouts.
3. Wave 2: ship durability/security hygiene: gdrive API streaming/safe overwrite, Cloud Run timeout cancel/late-marker check, retention scheduling, obsreport URL/CI, trackers SHA pin.
4. Wave 3: repo gates: sports-data-collector ruff/coverage, sports-obsreport CI, khelsutra web E2E CUJ in CI, refresh engine vendored SPA bundle.
5. Wave 4: UX fixes: khelsutra job resume/progress, upload filename, hide demo grid while processing, HTML 5xx error cleanup, WhatsApp file share.

6. Wave 5: structural cleanup: close/re-cut `#390` main.py split after behavior fixes, then remove shim cycles and continue DB/module splits opportunistically.

I'd keep owner-gated items flagged only: Cloudflare cleanup, public launch gates, 38GB golden-video migration, P0 name cleanup, ship-gate target, and legal/licensing decisions.

31. user (2026-07-02T06:29:29.484Z)

LGTM! Please create a project plan based on the template you have identified and make all the doc editions and updates in a single first docs only PR and then follow up with the execution plan wave wise. Do ensure that the test coverage and always stays up and if you are changing the SPA and the UI modules then do ensure to have a screencast duly created and validated as in khelsutra-screencasts. Is that possible?

32. assistant (2026-07-02T06:29:42.299Z)

Yes, possible. I'll do this as a docs-only bootstrap PR first: a tracked project plan, index/register updates, and an explicit wave plan that bakes in coverage preservation plus the screencast requirement for any later SPA/UI wave. Then I'll stop before code waves so that first PR stays cleanly docs-only.

33. assistant (2026-07-02T06:29:48.432Z)

I'm using the engineering documentation skill for this because the next artifact is a durable repo plan, not just a chat plan. I'll keep it faithful to the repo's tracked-project format rather than inventing another planning shape.

34. assistant (2026-07-02T06:30:02.848Z)

I'm going to make the first PR in the engine repo, because that is where the audit report already landed and where `PROJECT\_DOC\_TEMPLATE.md`, `DOC\_STATUS.md`, and the doc index live. The sibling repo work will be referenced from the project doc, then executed as separate repo-specific PRs in the later waves.

35. assistant (2026-07-02T06:30:07.593Z)

The engine tree is clean and fetched. I'm branching explicitly from `origin/master` now, per the shared-checkout safety rule, then I'll make only docs edits on that branch.

36. assistant (2026-07-02T06:30:31.241Z)

I'm drafting the plan as the control plane for the cleanup, not as another audit dump. It will record what is already complete after syncing, what I dispute or downgrade, and the one-PR-per-deliverable wave order, including the UI screencast rule as a locked decision.

37. assistant (2026-07-02T06:32:22.911Z)

The new project doc is in place. I'm wiring it into the repo's living doc system now: README index, DOC_STATUS lifecycle row, NEXT_STEPS pointer, and a small audit-note so future readers don't treat the snapshot as the live tracker.

38. assistant (2026-07-02T06:33:00.087Z)

The doc wiring is done. I'm validating the docs now with the repo's markdown link checker and reviewing the diff to catch any accidental non-doc churn before I commit.

39. assistant (2026-07-02T06:33:11.747Z)

The link checker is clean. I noticed the diff stat from Git doesn't show the new file until it's tracked, so I'm doing the final whitespace/status checks, then I'll stage only the six docs files explicitly.

40. assistant (2026-07-02T06:34:18.057Z)

I've got the thread back after the context handoff. The docs-only plan is already drafted and link-checked; I'm doing the safe pre-commit checks now, then I'll push the branch and open the PR.

41. user (2026-07-02T06:34:19.052Z)

Since your context is getting full, do you think you can submit the docs only PR, set up a very good plan and then I should use a new session for this work? Can you help me give a prompt which I can have the other session start where you left off?

42. assistant (2026-07-02T06:34:46.112Z)

Yes, that's exactly the right shape: I'll land the docs-only PR as the clean handoff artifact, then give you a strong "start here" prompt for a fresh session. I'm going to commit only the six doc edits plus the new plan file, with the repo's safe branch/stage protocol.

43. assistant (2026-07-02T06:34:50.583Z)

The plan file is in the repo's project-doc style, and it explicitly records the completed audit items, the disputed/reclassified ones, and the UI screencast rule. I'm staging only the named doc paths now.

44. assistant (2026-07-02T06:35:03.716Z)

Staged cleanly, with only Git's line-ending normalization warnings on Markdown files. I'm checking the staged patch and markdown links one more time before committing.

45. assistant (2026-07-02T06:35:08.218Z)

The staged docs patch is small and link-clean: six files, 226 insertions, no whitespace errors. I'm committing it now, then I'll push and open the PR.

46. assistant (2026-07-02T06:35:15.629Z)

Commit is made on the docs branch. I'm doing the branch-diff safety check against `origin/master` before pushing, so the PR contains only this docs-only commit.

47. assistant (2026-07-02T06:35:18.829Z)

The branch diff is exactly one commit on top of `origin/master`, and the working tree is otherwise clean. Pushing now and then I'll create the PR with the verification notes.

48. assistant (2026-07-02T06:35:28.696Z)

Branch is on GitHub. I'm opening the docs-only PR now, with the link-check and staged-diff checks called out so the next session has a crisp starting point.

49. assistant (2026-07-02T06:35:47.250Z)

PR #423 is open. Before I wrap, I'm going to refresh the lightweight shared memory handoff so the new session can ramp from the PR and the tracked plan rather than replaying this whole audit pass.

50. assistant (2026-07-02T06:35:55.044Z)

I found the shared memory files. The current-state entry still reflects the pre-reconciliation "six-wave approved" version; I'm updating that to the actual state now: docs PR open, completed items reconciled, and next work begins from the tracked plan.

51. assistant (2026-07-02T06:36:08.161Z)

The old live-state line is a single dense paragraph, so I'm replacing it with a shorter, current checkpoint instead of stacking more lore on top of it. That keeps the next session's first read

honest.

52. assistant (2026-07-02T06:36:44.422Z)

Current-state is now accurate and points straight at PR #423 plus the new plan. I'm adding a concise 2026-07-02 handoff section as the next-session ?you are here? anchor.

53. assistant (2026-07-02T06:37:08.742Z)

The handoff has an old long ?last updated? line, so I'm making the front matter current and leaving the older sections below as history. The fresh 2026-07-02 section will be the new session's real ramp-up point.

54. assistant (2026-07-02T06:37:45.191Z)

The handoff now names the docs PR, the reconciled repo heads, the still-open `R1-27`, and the first Wave 1 rows. I'll quickly check the PR/check status and repo state, then I'll give you the handoff prompt for the new session.

55. assistant (2026-07-02T06:37:53.437Z)

PR #423 is mergeable and every CI job is green except the full offline test suite, which is still running. Since this is the docs gate for the handoff, I'm going to wait for that one result rather than leaving you with ?probably fine.?

56. assistant (2026-07-02T06:38:40.717Z)

All CI is green on the docs-only PR, including the full offline mocked suite. I'm adding that final CI result to the handoff note, then I'll stop there rather than merging under your feet.

57. user (2026-07-02T06:38:57.127Z)

you can merge it!

58. assistant (2026-07-02T06:39:12.650Z)

Great, I'll merge it now. I'm doing one last PR-state check, then I'll squash-merge/delete the remote branch and fast-forward the local checkout back to `master`.

59. assistant (2026-07-02T06:39:17.046Z)

All checks are green and the PR is mergeable; the branch still contains only the docs-plan commit. Merging it now.

60. assistant (2026-07-02T06:39:36.610Z)

Merged. GitHub fast-forwarded the local checkout to the squash commit as part of the merge command; I'm checking the branch/status now and will put the repo back on `master` at origin head if needed.

61. assistant (2026-07-02T06:39:46.422Z)

The repo is back on clean `master` at `origin/master`, with squash commit `e517475` from PR #423. I'm making the handoff/current-state wording match reality now: merged, not open.

62. assistant (2026-07-02T06:40:43.324Z)

Merged PR #423: [docs: track khelsutra-guru health remediation plan](<https://github.com/Khelsutra/badminton-highlight-indexer/pull/423>)

It landed on `master` as `e517475`. CI was fully green, including the full offline mocked suite. Local engine repo is clean and on `master` at `origin/master`. I also refreshed the OneDrive handoff/current-state so the next session has the right ?start here? state.

Use this as the fresh-session prompt:

```
```text
```

Continue the khelsutra-guru health remediation work.

First, obey the repo freshness rule: open C:\Users\avidu\Projects\khelsutra-guru, then for every sub-repo you touch run `git pull --ff-only` before reading. Also read C:\Users\avidu\OneDrive\Documents\claude-sync\memory\khelsutra-guru\session-handoff.md and current-state.md.

The docs-only bootstrap PR is merged:

- Engine PR #423, commit e517475
- Live tracker: badminton-highlight-indexer/docs/KHELSUTRA\_GURU\_HEALTH\_REMEDIATION\_PLAN.md
- Start from §7 of that plan.

Current task: execute the plan wave-wise, one small PR per §7 row. Begin with Wave 1 in badminton-highlight-indexer:

1. P2: fix #340 idempotent processing / Gemini re-bill risk.
2. Then P3 source caching, P4 ffmpeg timeouts, P5 scratch/retention, P6 real gcs.py mypy fix.

Important constraints:

- Branch from remote explicitly: `git fetch origin` then `git checkout -b <branch> origin/master`.
- Re-verify branch + HEAD before staging/commit.
- Stage explicit paths only.
- Keep coverage at/above repo floor; run repo-appropriate lint/type/tests and record verification in PR.
- If touching khelsutra SPA/UI or UI modules, create and validate screenshot/screencast evidence using the existing khelsutra-screencast convention.
- sports-data-collector has a pre-existing dirty `.gitignore` hunk; do not clobber it.
- R1-27 is still open despite PR #422: `python -m mypy backend training --no-incremental` still fails at backend/storage/gcs.py:31 when google-cloud-storage is installed.

```
```
```

Nice clean handoff point. The next session can start doing code instead of archaeology.